

Detailed Line-by-Line Explanation of OpenMP Reduction Program

This document explains every important line of the uploaded C++ OpenMP program. The code demonstrates: Sequential Minimum Calculation Parallel Minimum Calculation using OpenMP Reduction Sequential Maximum Calculation Parallel Maximum Calculation Sequential and Parallel Sum Sequential and Parallel Average The program compares execution times between sequential and parallel implementations.

Line	Code	Detailed Explanation
1	<code>#include <iostream></code>	Includes the input-output stream library. Used for cin and cout operations.
2	<code>#include <omp.h></code>	Includes the OpenMP library for parallel programming support.
3	<code>#include <vector></code>	Includes STL vector container support for dynamic arrays.
4	<code>#include <climits></code>	Provides constants such as INT_MAX and INT_MIN.
5	<code>using namespace std;</code>	Allows standard library names to be used without std:: prefix.
7	<code>//----MINIMUM----</code>	Comment indicating the minimum value section.
8	<code>void minimum(vector<int>& array)</code>	Defines a function to compute minimum element.
9	<code>{</code>	Beginning of minimum function.
10	<code>int min_val = INT_MAX;</code>	Initializes minimum value with the largest integer possible.
12	<code>double start = omp_get_wtime();</code>	Starts timer using OpenMP wall-clock timer.
14	<code>for (int i=0;i<array.size();i++)</code>	Sequential loop iterating through the array.
15	<code>{</code>	Begins loop body.
16	<code>if (array[i] < min_val)</code>	Checks if current element is smaller than current minimum.
17	<code>{</code>	Begins if block.
18	<code>min_val = array[i];</code>	Updates minimum value.
19	<code>}</code>	Ends if block.
20	<code>}</code>	Ends loop.
22	<code>double end = omp_get_wtime();</code>	Stores ending execution time.
24	<code>cout<<"\nMinimum (Sequential) : "<<min_val<<endl;</code>	Prints sequential minimum result.
25	<code>cout<<"Time : "<<end-start<<endl;</code>	Prints sequential execution time.
27	<code>int min_ele = INT_MAX;</code>	Initializes minimum variable for parallel version.
28	<code>start = omp_get_wtime();</code>	Starts timer for parallel execution.
30	<code>#pragma omp parallel for reduction(min:min_ele)</code>	Parallelizes loop and performs reduction using minimum operation.
31	<code>for (int i=0;i<array.size();i++)</code>	Parallel loop iterating through array elements.
32	<code>{</code>	Begins parallel loop body.
33	<code>if (array[i] < min_ele)</code>	Checks if current element is smaller.

34	{	Begins if block.
35	min_ele = array[i];	Updates local thread minimum.
36	}	Ends if block.
37	}	Ends parallel loop.
38	end = omp_get_wtime();	Stores ending time for parallel execution.
40	cout<<"\nMinimum (Parallel) : "<<min_val<<endl;	Prints parallel minimum result.
41	cout<<"Time : "<<end-start<<endl;	Prints parallel execution time.
42	}	Ends minimum function.
44	//----MAXIMUM----	Comment indicating maximum section.
45	void maximum(vector<int>& array)	Defines maximum value function.
46	{	Begins maximum function.
47	int max_val = INT_MIN;	Initializes maximum with smallest integer value.
49	double start = omp_get_wtime();	Starts timer.
51	for (int i=0;i<array.size();i++)	Sequential traversal loop.
52	{	Begins loop body.
53	if (array[i] > max_val)	Checks if current element is larger than maximum.
54	{	Begins if block.
55	max_val = array[i];	Updates maximum value.
56	}	Ends if block.
57	}	Ends loop.
59	double end = omp_get_wtime();	Stores ending time.
61	cout<<"\nMaximum (Sequential) : "<<max_val<<endl;	Prints sequential maximum.
62	cout<<"Time : "<<(end-start)<<endl;	Prints sequential timing.
64	int max_ele = INT_MIN;	Initializes variable for parallel maximum.
65	start = omp_get_wtime();	Starts timer for parallel version.
67	#pragma omp parallel for reduction(max:max_ele)	Runs loop in parallel with maximum reduction.
68	for (int i=0;i<array.size();i++)	Parallel loop.
69	{	Begins loop body.
70	if (array[i] > max_ele)	Checks for larger value.
71	{	Begins if block.
72	max_ele = array[i];	Updates local maximum.
73	}	Ends if block.
74	}	Ends parallel loop.
75	end = omp_get_wtime();	Stores ending time.
77	cout<<"\nMaximum (Parallel) : "<<max_val<<endl;	Prints parallel maximum.

78	cout<<"Time : "<<(end-start)<<endl;	Prints timing result.
79	}	Ends maximum function.
81	//---SUM---	Comment indicating sum section.
82	void sum(vector<int>& array)	Defines sum function.
83	{	Begins sum function.
84	long long total = 0;	Initializes total sum variable.
85	double start = omp_get_wtime();	Starts timer.
86	for(int i=0;i<array.size();i++)	Sequential loop for summation.
87	{	Begins loop body.
88	total = total + array[i];	Adds current element to total.
89	}	Ends loop.
90	double end = omp_get_wtime();	Stores end time.
91	cout<<"\nSum (Sequential) : "<<total<<endl;	Prints sequential sum.
92	cout<<"Time : "<<(end-start)<<endl;	Prints timing.
94	total = 0;	Resets total for parallel version.
95	start = omp_get_wtime();	Starts timer.
96	#pragma omp parallel for reduction(+:total)	Parallel reduction using addition.
97	for (int i=0;i<array.size();i++)	Parallel loop for summation.
98	{	Begins loop body.
99	total = total + array[i];	Adds values in parallel.
100	}	Ends loop.
101	end = omp_get_wtime();	Stores ending time.
102	cout<<"\nSum (Parallel) : "<<total<<endl;	Prints parallel sum.
103	cout<<"Time : "<<(end-start)<<endl;	Prints timing result.
104	}	Ends sum function.
106	//---AVERAGE---	Comment indicating average section.
107	void average(vector<int>& array)	Defines average function.
108	{	Begins average function.
109	long long total = 0;	Initializes sum variable.
110	double start = omp_get_wtime();	Starts timer.
111	for(int i=0;i<array.size();i++)	Sequential loop.
112	{	Begins loop body.
113	total = total + array[i];	Adds current value.
114	}	Ends loop.
115	double end = omp_get_wtime();	Stores ending time.

116	cout<<"\nAverage (Sequential) : "<<(double)total/array.size()<<"\n";	Calculates and prints average.
117	cout<<"Time : "<<(end-start)<<endl;	Prints execution time.
119	total = 0;	Resets total for parallel version.
120	start = omp_get_wtime();	Starts timer.
121	#pragma omp parallel for reduction(+:total)	Parallel reduction for summation.
122	for (int i=0;i<array.size();i++)	Parallel traversal loop.
123	{	Begins loop body.
124	total = total + array[i];	Adds values in parallel.
125	}	Ends loop.
126	end = omp_get_wtime();	Stores ending time.
127	cout<<"\nAverage (Parallel) : "<<(double)total/array.size()<<"\n";	Calculates and prints parallel average.
128	cout<<"Time : "<<(end-start)<<endl;	Prints execution time.
129	}	Ends average function.
131	int main()	Main function where program execution begins.
132	{	Begins main function.
133	int N;	Stores number of elements.
134	int MAX = 1000;	Defines maximum random value.
135	cout<<"Enter number of elements : ";	Prompts user for input size.
136	cin>>N;	Reads number of elements.
138	vector<int> array(N);	Creates vector of size N.
140	for (int i=0;i<N;i++)	Loop to initialize array.
141	{	Begins loop body.
142	array[i] = rand() % MAX;	Generates random values between 0 and 999.
143	//cin>>array[i];	Alternative commented line for manual input.
144	}	Ends loop.
146	minimum(array);	Calls minimum function.
147	maximum(array);	Calls maximum function.
148	sum(array);	Calls sum function.
149	average(array);	Calls average function.
151	return 0;	Indicates successful program completion.
152	}	Ends main function.

Overall Concept:

The program demonstrates how OpenMP reduction operations can improve performance for operations like minimum, maximum, sum, and average on large datasets. Key OpenMP concepts used:
reduction(min:variable) for minimum calculation **reduction(max:variable)** for maximum calculation

reduction(+:variable) for summation Reduction combines results from multiple threads safely and efficiently.